



Basic Information

- Experiment No: 15
- Date: 18/09/2026
- Title: Implementation of Runtime Polymorphism using Method Overriding in Employee Salary Management



Objective / Aim

To write a Java program that demonstrates runtime polymorphism using inheritance, method overriding, constructors, and a base class reference to display the salary details of regular and contract employees.



System Requirements

- Hardware: Standard PC setup (Core i3/i5 Processor, 4GB/8GB RAM).
- Software: Windows/Linux/macOS, Java Development Kit (JDK 17 or JDK 21), IntelliJ IDEA / Eclipse / NetBeans / VS Code.



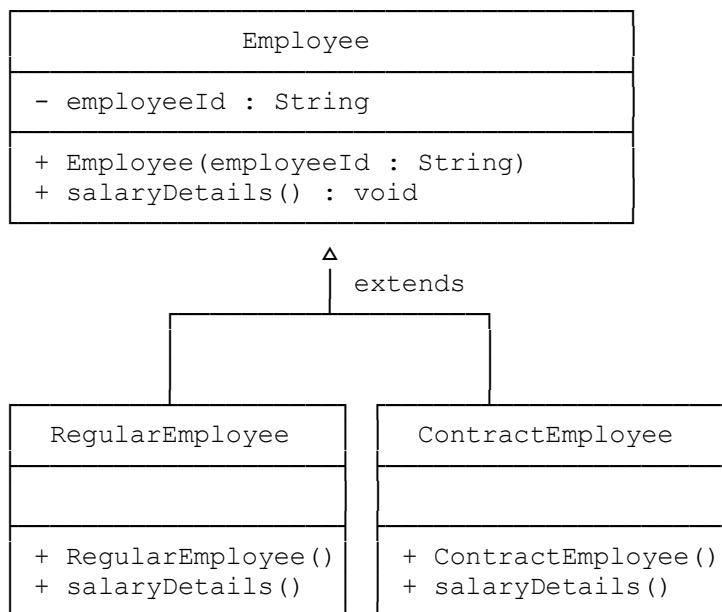
Algorithm / Logic Description

1. Start the program.
2. Create a base class named `Employee` with an employee ID.
3. Define a constructor to initialize the employee ID.
4. Define the `salaryDetails()` method in the `Employee` class.
5. Create a derived class `RegularEmployee` that inherits from `Employee`.
6. Use the `super` keyword to call the parent class constructor.
7. Override the `salaryDetails()` method to calculate the regular employee's total salary.
8. Create another derived class `ContractEmployee` that inherits from `Employee`.
9. Override the `salaryDetails()` method to calculate the contract employee's total salary.

10. Create a `Scanner` object to read the employee ID.
11. Declare an `Employee` reference variable.
12. If the employee ID starts with `R` or `r`, create a `RegularEmployee` object.
13. If the employee ID starts with `C` or `c`, create a `ContractEmployee` object.
14. If the employee ID is invalid, display "Invalid Employee Id".
15. Call `salaryDetails()` using the `Employee` reference.
16. Demonstrate runtime polymorphism through method overriding.
17. Stop the program.



UML Class Diagram



Source Code

```
package record;

import java.util.Scanner;
//Employee class
class Employee {
```

```

String employeeId;

Employee(String employeeId) { // constructor
    this.employeeId = employeeId;
}

void salaryDetails() {
    System.out.println("Salary details not available");
}
}

//RegularEmployee class inherits Employee class
class RegularEmployee extends Employee {

    RegularEmployee(String employeeId) {
        super(employeeId);
    }

    @Override
    void salaryDetails() {
        int basicPay = 25000;
        int hra = 15000;
        int ta = 5000;
        int total = basicPay + hra + ta;

        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basicPay+ " HRA: " + hra+ " T.A: " + ta + " Total
Amount: " + total);
    }
}

//ContractEmployee inherits Employee
class ContractEmployee extends Employee {
    //constructor
    ContractEmployee(String employeeId) {
        super(employeeId);
    }

    @Override
    void salaryDetails() {
        int basicPay = 12000;
        int ta = 3000;
        int total = basicPay + ta;

        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basicPay+ " T.A: " + ta + " Total Amount: " +
total);
    }
}

```

```

public class EmployeeDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee Id: ");
        String id = sc.nextLine();

        Employee emp;

        // R = Regular Employee
        // C = Contract Employee
        if (id.charAt(0) == 'R' || id.charAt(0) == 'r') {
            emp = new RegularEmployee(id);
        }
        else if (id.charAt(0) == 'C' || id.charAt(0) == 'c') {
            emp = new ContractEmployee(id);
        }
        else {
            System.out.println("Invalid Employee Id");
            sc.close();
            return;
        }

        // Runtime polymorphism
        emp.salaryDetails();

        sc.close();
    }
}

```

Compilation & Execution Commands

Save the file as:

```
EmployeeDemo.java
```

Compile the code:

```
javac -d . EmployeeDemo.java
```

Run the compiled bytecode:

```
java record.EmployeeDemo
```



Output Screen

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Progr  
Enter Employee Id: R101  
Salary Details:  
Basic Pay: 25000 HRA: 15000 T.A: 5000 Total Amount: 45000  
  
Process finished with exit code 0
```